Outcome-Aware Agents: From Token Prediction to Action Consequence Modeling

Mallesh Madapathi
ThinkingDBx Pvt. Ltd., Hyderabad, India
mallesh@thinkingdbx.com

Abstract

Large language models are increasingly deployed as agents that execute code, modify databases, and operate production systems, yet their training objective contains no explicit signal about what their actions do. We argue that agents need an internal forward model that predicts the consequences of a candidate action before it is executed, and we study one such mechanism, the Action-Conditioned Latent Structural Causal Model (AC-LSCM): latent factors with per-factor mechanisms over a learned sparse graph, actions applied as do-interventions, and a planner that rejects candidates whose simulated multi-step outcome is unsafe. This version corrects an earlier one. We show that the originally specified model learned an empty causal graph in every run, and that the agent task behind its headline result (36× fewer safety violations than a Transformer) tests only the direct effect of an action and placed unsafe values outside the training range; the Transformer's violations there drop from 15.6% to 2.0% when the values are moved in range, and we withdraw that claim. A simplified AC-LSCM, without the acyclicity term (unnecessary for lagged dynamics) and the contrastive hinge, with exact abduction and a straight-through graph mask, has the lowest one-step and counterfactual prediction error of all models on five synthetic graphs. On a new consequence task in which safety depends on downstream effects, it matches a noise-free oracle (4.7% violations versus 7.3% for the best baseline) when its graph keeps all true edges, but only with ground-truth counterfactual supervision; without that privileged signal it is on par with the best baseline (6.9%). Pruning a single weak edge into the safety factor raises violations to 8.4%, and the do-operator itself has no measurable effect. We report all of these results, including the negative ones, and release code and per-seed results.

Keywords: LLM agents, world models, causal representation learning, structural causal models, safe action selection, counterfactuals.

1 Introduction: Agents That Do Not Know What They Are Doing

Consider a code agent that generates and executes a database migration. The migration is syntactically valid, follows recognizable patterns from training data, and matches the user's intent as stated in natural language. It also silently drops a foreign key constraint that downstream services depend on. The model did not predict this consequence, because nothing in how it was trained asked it to predict consequences. It was trained to predict tokens.

This failure pattern matters more as large language models (LLMs) are wired into agent frameworks (tool-using assistants, autonomous coders, browser agents, data engineering copilots) that take actions which change real systems. The pre-training objective remains $P(w_t \mid w_{<t})$, and

nothing in that objective directly rewards an accurate model of what happens after an action is taken.

The thesis of this paper is that an AI system that takes actions should be able to predict, in advance, what those actions will cause, and should use that prediction to filter, rank, or reject candidate actions before execution. In other words, agents need an explicit internal *forward model* of action consequences, coupled to the action-selection rule.

We study one such mechanism, the Action-Conditioned Latent Structural Causal Model (AC-LSCM), together with a planning loop that uses it for outcome-aware action selection. AC-LSCM draws on three established lines of work: action-conditioned latent world models [6–8], causal representation learning [12, 13, 17], and sparse graph learning over latent dynamics [2, 19, 20]. It represents the world state as a small set of latent factors whose transitions follow a learned sparse graph, and implements actions as structural interventions in the sense of Pearl's do-operator [15].

What this version reports. An earlier version of this work [14] reported that AC-LSCM reduced safety violations roughly 36× relative to a Transformer baseline on an agent planning task. Re-examining that result, we found two problems that change its interpretation, and this version reports them together with corrected experiments:

1. In every run of the originally specified model, the learned causal graph was *empty*: the sparsity and acyclicity penalties drove every edge off, so at evaluation time the transition model ignored the current state entirely (Section 9.1).

2. The original agent task only tested the *direct* effect of an action on the factor it sets, and placed the unsafe action values outside the range seen in training. A state-blind model solves it, and the baseline's failures there are failures of extrapolation, not of causal reasoning (Section 9.2).

Contributions. (1) An architecture and planning loop for outcome-aware action selection, in which actions enter the forward model as interventions and candidate actions are filtered on simulated multi-step outcomes. (2) A diagnosis of why the originally specified model failed to learn structure, and a simplified AC-LSCM that removes the components the ablations identified as harmful, drops an acyclicity constraint that is unnecessary for lagged dynamics, and uses exact abduction and a straight-through graph mask. (3) A *consequence task* in which safety and goal are defined on factors downstream of the intervened one and judged on the realized multi-step outcome of the ground-truth system, so that a forward model must propagate an intervention through the causal graph. (4) Experiments on synthetic structural causal models comparing AC-LSCM with three forward-model baselines under one shared planner. The simplified model has the lowest one-step and counterfactual error on all graphs and matches an oracle planner when its graph keeps every true edge; without privileged counterfactual supervision it is on par with the best baseline, and the do-operator itself has no measurable effect. We report these negative results in full.

Scope. All experiments use synthetic structural causal models with up to 20 factors. We do not evaluate on language benchmarks or LLM-scale models. Section 11 states what the evidence does and does not establish.

2 The Agent Problem with Token Prediction

2.1 Why next-token training is not an action model

An autoregressive language model is trained to maximize $\log P(w_t \mid w_{<t})$ over a corpus. When such a model is deployed as an agent, an action a_t is another token sequence sampled from this distribution. Nothing in the objective requires an explicit representation of the world state the action will modify, or of the state that will result.

This is often sufficient when both the action and its consequences are well represented in training data and the consequences are themselves textual. It is insufficient in three situations that are common in agent deployments: when the action changes a system outside the text stream (a filesystem, a database, a browser DOM, a physical actuator); when the training corpus does not contain the specific configuration the agent is acting on; and when the right action depends on the resulting state rather than on surface similarity to training contexts. The resulting failures are not hallucinations in the usual sense. They are actions taken without a model of their outcome.

2.2 Concrete failure modes

Illustrative examples include code agents that emit destructive shell commands when a task superficially resembles a cleanup, SQL agents that propose joins matching familiar templates but producing wrong results on the actual schema, browser agents that submit a plausible-looking form on the wrong account, and pipeline agents that recommend transformations whose effect on downstream consumers they never represented. Benchmarks that emulate tool execution to expose such risks [16] document that these failures are frequent in current LM agents. In each case the agent selected an action by pattern-matching rather than by consulting a model of what the action would do to the current system state.

2.3 What an outcome-aware agent needs

An agent that anticipates the consequences of its actions needs four components: a compact representation $\mathbf{z}_t$ of the current state; a transition model $p(\mathbf{z}_{t+1} \mid \mathbf{z}_t, a_t)$ that predicts the next state given an action; predicates and a value function that score predicted outcomes against goals and constraints; and a selection rule that uses simulated outcomes to choose, defer, or reject candidate actions. Planning with learned models [7, 18], shielding in safe reinforcement learning [1], and recent world-model-based LLM agents [5, 9] all instantiate parts of this recipe. The question we study is how the transition model should be structured so that it captures the *consequences* of interventions, including indirect ones, rather than correlations between action tokens and successor tokens.

3 Pearl's Hierarchy as a Diagnostic

Pearl [15] distinguishes three levels of reasoning. Association, $P(y \mid x)$, concerns what is observed to co-occur. Intervention, $P(y \mid \mathrm{do}(x))$, concerns what happens when a variable is set by an external agent. Counterfactuals, $P(y_x \mid x', y')$, concern what would have happened in a specific realized case under a different action.

For an agent, the relevant distinction is between "users who run command X tend to be in situation Y afterward" (association) and "if I run command X, the system will be in situation Y afterward" (intervention). Two caveats keep this distinction honest. First, when an agent conditions on its own action and the full state, and nothing unobserved influences both the action choice and the outcome, the conditional and interventional distributions coincide; an action-conditioned model

Dimension	Token-prediction agent	Outcome-aware agent (AC-LSCM)
What it models	Token distributions in context	State transitions under actions
Action handling	Action tokens emitted from context	Structural intervention on the state model
Causal level	Association (by default)	Intervention and counterfactual
Failure mode	Plausible action, wrong outcome	Action rejected if its simulated outcome is unacceptable

Table 1: Two agent paradigms.

trained on such data can in principle learn $P(\cdot \mid \mathrm{do}(a))$ without special machinery. Second, LLMs trained or fine-tuned on interaction data are not restricted to the associational level. Benchmarks such as CLadder [10] and CounterBench [4] nonetheless show that current LLMs perform poorly on formal interventional and counterfactual queries.

What structural interventional semantics adds is *modularity*: if each factor has its own mechanism and an action replaces exactly the mechanism of the factor it targets, then the effect of an intervention on one factor can be propagated to its descendants through mechanisms that were learned from other data. Whether this inductive bias helps in practice is the empirical question this paper addresses. Table 1 summarizes the contrast.

4 The AC-LSCM Architecture

4.1 State representation

The world state is represented as K latent factors,

$$\mathbf{z}_t = (z_t^1, z_t^2, \ldots, z_t^K), \qquad z_t^i \in \mathbb{R}^{d_i}, \tag{1}$$

each intended to track one mechanism of the environment. In our experiments $d_i = 1$. Identifiability of such factors is not guaranteed in general (Section 11); in our setting each action targets a known factor index, which anchors the latent coordinates to the intervention targets, as in work on causal representation learning from interventions [3, 13].

4.2 Structural transitions

Transitions are governed by per-factor structural equations parameterized by neural networks f_i:

$$z_{t+1}^i := f_i(\mathrm{Pa}(z^i)_t,\ a_t^i) + \epsilon_t^i, \qquad i \in \{1, \ldots, K\}, \tag{2}$$

where $\mathrm{Pa}(z^i)_t \subseteq \mathbf{z}_t$ is the parent set of factor i under a learned adjacency matrix $A \in \{0,1\}^{K \times K}$, a_t^i is the component of the action that targets factor i (null if the action does not touch it), and ϵ_t^i is exogenous noise. Parents are taken at time t and children at $t+1$, so the unrolled graph is acyclic for any A. An acyclicity constraint on A is therefore only needed if one also allows instantaneous (same-step) edges; we return to this point in Section 9.1. The adjacency is parameterized by logits Θ, sampled with a Gumbel-sigmoid relaxation during training, and thresholded at $\sigma(\Theta) > 0.5$ at evaluation. In the original model the training-time sample is soft, so the masked parent value $\tilde{A}_{ij} z_t^j$ can be rescaled by f_i; in the simplified model it is a hard sample with a straight-through gradient, so training and evaluation use the same binary graph.

4.3 Action as intervention, not concatenation

When a_t^i is non-null, the transition for factor i does not receive its parents: the natural causes are severed and the intervention value is substituted, $z_{t+1}^i := f_i(\varnothing, a_t^i)$, with the network expected to learn $f_i(\varnothing, v) \approx v$. This is the do-operator made explicit in the computation graph. A generic sequence model instead concatenates an action embedding to its input and must learn from data which inputs to ignore.

4.4 Trainable modules

An encoder $q_\phi(\mathbf{z}_t \mid \mathbf{x}_t)$ maps observations to latent factors; a decoder $p_\theta(\mathbf{x}_t \mid \mathbf{z}_t)$ reconstructs observations; and the transition operator $p_\psi(\mathbf{z}_{t+1} \mid \mathbf{z}_t, \mathrm{do}(a_t))$ implements Equation (2) with mean given by the f_i and unit variance. The adjacency logits Θ are learned jointly.

5 Training Objective

The full objective as originally specified is

$$\mathcal{L} = \beta_1 \mathcal{L}_{\text{recon}} + \beta_2 \mathcal{L}_{\text{trans}} + \beta_3 \mathcal{L}_{\text{graph}} + \beta_4 \mathcal{L}_{\text{cf}}. \tag{3}$$

The reconstruction loss anchors the latent prediction to the next observation,

$$\mathcal{L}_{\text{recon}} = \big\|\mathbf{x}_{t+1} - \hat{\mathbf{x}}_{t+1}\big\|_2^2, \qquad \hat{\mathbf{x}}_{t+1} = \mathrm{dec}_\theta\big(\bar{f}(\mathrm{enc}_\phi(\mathbf{x}_t), \mathrm{do}(a_t))\big), \tag{4}$$

where $\bar{f}$ stacks the f_i. The transition loss matches the predicted next state to the encoder posterior of the observed next state,

$$\mathcal{L}_{\text{trans}} = \mathbb{E}\Big[D_{\text{KL}}\big(q_\phi(\mathbf{z}_{t+1} \mid \mathbf{x}_{t+1}) \,\big\|\, p_\psi(\mathbf{z}_{t+1} \mid \mathbf{z}_t, \mathrm{do}(a_t))\big)\Big]. \tag{5}$$

The graph loss combines sparsity with the NOTEARS acyclicity function [20],

$$\mathcal{L}_{\text{graph}} = \lambda_1 \|\tilde{A}\|_1 + \lambda_2\big(\,\mathrm{tr}(e^{\tilde{A}\odot\tilde{A}}) - K\big), \tag{6}$$

where $\tilde{A}$ is the relaxed adjacency. The DAGMA log-determinant function [2] is an alternative for the second term. The counterfactual loss uses ground-truth counterfactual latents $\mathbf{z}_{t+1}^{\text{cf}}$, available in simulation:

$$\mathcal{L}_{\text{cf}} = \big\|\tilde{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}^{\text{cf}}\big\|_2^2 + \max\big(0,\ \gamma - \|\tilde{\mathbf{z}}_{t+1} - \hat{\mathbf{z}}_{t+1}\|_2^2\big), \tag{7}$$

where $\tilde{\mathbf{z}}_{t+1}$ is the model's counterfactual prediction under an alternative action $\tilde{a}_t$ (Section 6) and $\hat{\mathbf{z}}_{t+1}$ is its factual prediction. The hinge term was intended to keep factual and counterfactual predictions from collapsing together.

Simplified AC-LSCM. The ablations of the original model (Section 9.1) and the lagged-graph argument above lead to a simplified objective, which is the configuration we recommend: (i) drop the acyclicity term ($\lambda_2 = 0$), since the lagged graph is acyclic by construction; (ii) reduce the sparsity weight from $\lambda_1 = 0.1$ to $\lambda_1 = 0.01$ and initialize the adjacency logits at $+2$ (dense) instead of 0, so that edges are pruned from a dense graph rather than having to be switched on against a penalty; (iii) remove the hinge term in Equation (7); (iv) use exact abduction (Section 6); and (v) sample the adjacency with a hard straight-through Gumbel-sigmoid during training. Everything else, including all network sizes and optimization settings, is unchanged. Changes (i)–(iv) follow from the first-version ablations and the lagged-graph argument and were fixed after a single 30-epoch pilot run; change (v) was added after the first full runs of (i)–(iv), described in Section 9.4, and was selected on held-out prediction error, not on agent-task performance.

Algorithm 1 Outcome-aware action selection

Require: observation $\mathbf{x}_t$, candidate actions $\mathcal{A}_t$, horizon H, safety predicate S, goal predicate G, value function V

$\mathbf{z}_t \leftarrow \mathrm{enc}_\phi(\mathbf{x}_t)$; $\quad \mathcal{C} \leftarrow \emptyset$
for each $a \in \mathcal{A}_t$ **do**
  $\mathbf{z}^{(a)}_{t+1} \leftarrow \bar{f}(\mathbf{z}_t, \mathrm{do}(a))$; $\quad \mathbf{z}^{(a)}_{t+k} \leftarrow \bar{f}(\mathbf{z}^{(a)}_{t+k-1}, \varnothing)$ for $k = 2, \ldots, H$
  $\hat{\mathbf{x}}^{(a)}_{t+k} \leftarrow \mathrm{dec}_\theta(\mathbf{z}^{(a)}_{t+k})$ for $k = 1, \ldots, H$
  if $S(\hat{\mathbf{x}}^{(a)}_{t+k})$ holds for all k **then**
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{a\}$
  end if
end for
if $\mathcal{C} = \emptyset$ **then**
  return DEFER
end if
return $\arg\max_{a \in \mathcal{C}} \; [v_a + \mathbf{1}[v_a > \tau_g]]$, where $v_a = \max_k V(\hat{\mathbf{x}}^{(a)}_{t+k})$

6 Counterfactual Inference

For a query "we observed $(\mathbf{x}_t, a_t, \mathbf{x}_{t+1})$; what would $\mathbf{x}_{t+1}$ have been under action $\tilde{a}_t$?", the model follows Pearl's three steps.

Abduction. Under the additive-noise form of Equation (2), the realized noise is recovered as the residual

$$\hat{\epsilon}_t = \mathrm{enc}_\phi(\mathbf{x}_{t+1}) - \bar{f}(\mathrm{enc}_\phi(\mathbf{x}_t), \mathrm{do}(a_t)). \tag{8}$$

Intervention. Replace a_t by $\tilde{a}_t$, holding the mechanisms and the inferred noise fixed. *Prediction.* Roll the structural equations forward and decode:

$$\tilde{\mathbf{z}}_{t+1} = \bar{f}(\mathrm{enc}_\phi(\mathbf{x}_t), \mathrm{do}(\tilde{a}_t)) + m \odot \hat{\epsilon}_t, \qquad \tilde{\mathbf{x}}_{t+1} = \mathrm{dec}_\theta(\tilde{\mathbf{z}}_{t+1}). \tag{9}$$

In the original model $m = \mathbf{1}$. With *exact abduction*, $m_i = 0$ for every factor that is intervened on in either a_t or $\tilde{a}_t$: an intervention overrides the exogenous noise of the factor it sets, so the residual for that factor is either meaningless (in the factual world) or must not be added (in the counterfactual world). Reusing the abducted noise for all other factors is what makes the prediction counterfactual rather than merely interventional.

7 The Outcome-Aware Planning Loop

Algorithm 1 shows how the forward model is used for action selection. For each candidate action, the model encodes the current observation, applies the action as an intervention, rolls the transition forward for H steps without further actions, and decodes the predicted observations. Safety and goal predicates are evaluated on these decoded predictions, which is the one space shared by every model we compare. Candidates whose simulated trajectory violates the safety predicate are removed; if none remain, the agent defers. Among the remaining candidates the agent picks the one with the highest predicted value, with a bonus for satisfying the goal predicate.

Safety filtering therefore happens before execution, on simulated outcomes rather than on the text of the action; deferral is a first-class output; and the difference between predicted and realized outcomes after execution is a training signal for the forward model.

8 Experimental Setup

Environments. We generate structural causal models with known ground-truth graphs and mechanisms: chain, fork, and collider graphs with $K = 10$ factors, and Erdős–Rényi (ER) graphs with $K = 10$ and $K = 20$. Mechanisms are lagged and additive-noise (Appendix A); observations are noisy copies of the latent state. Training data is one trajectory of $N = 10{,}000$ steps in which, at each step with probability 0.5, one random factor is set by $\mathrm{do}(z^i = v)$ with $v \sim U(-1.5, 1.5)$. Each transition is paired with a ground-truth counterfactual that reuses the same noise under a different random intervention.

Models. Three baselines share AC-LSCM's encoder and decoder and differ in the transition. *Vanilla VAE* uses one MLP on the concatenated latent and action. *Dreamer-style* uses a recurrent (GRU) transition cell. *Small Transformer* treats the scalars of $(\mathbf{x}_t, a_t)$ as a token sequence and predicts $\mathbf{x}_{t+1}$; it is the closest of the three to a sequence-model forward model, but it is a forward model plugged into the same planner, not a language-model agent. We evaluate the *original* AC-LSCM as specified in the first version and the *simplified* AC-LSCM of Section 5, plus ablations of the latter.

Agent tasks. Both tasks use Algorithm 1 with predicates evaluated on decoded predictions unless stated otherwise. The *direct-effect task* is the task of the first version: 100 episodes on ER-10 with five candidate actions each; in 80 episodes one candidate sets the goal factor z^{K-1} above $\tau_g = 1.0$ and one sets the safety factor z^0 above $\tau_s = 1.5$; in the other 20 episodes every candidate sets z^0 above τ_s, so the correct response is to defer. Because each action sets the factor that the predicates read, the task only tests whether a model knows that $\mathrm{do}(z^i = v)$ yields $z^i \approx v$. Its unsafe values (1.6 to 2.5) also lie outside the training range of actions. We therefore also run an *in-range* variant with $\tau_s = 1.0$ and $\tau_g = 0.5$, which places all candidate values inside the training range. The *consequence task* (300 episodes on ER-10, identical for all models) is designed so that consequences must be propagated. Each candidate is a single intervention on a random factor with a value in the training range. Safety requires the safety factor z^9 to stay below $\tau_s = 0.4$ at every step of a three-step horizon; the goal is reached if the goal factor z^8 exceeds $\tau_g = 0.3$ at some step. In the ground-truth graph, z^9 has parents z^0, z^5, z^7 and ancestors $\{0, 2, 5, 7\}$, and z^8 has parent z^7; effects arrive one to three steps after the intervention, and goal and safety trade off through the shared ancestors z^2 and z^7. Outcomes are judged on the realized rollout of the true SCM under one shared noise draw. A quarter of the episodes contain no safe candidate; every other episode contains at least one safe candidate that reaches the goal, so the goal-rate ceiling is 0.75. As references, a planner that uses the noise-free true SCM from the noisy observation reaches 4.7% violations, a planner that knows only each action's direct effect (the intervened factor takes the intervened value, nothing else changes) reaches 31.7%, and one that predicts no effect at all reaches 45%.

Metrics and seeds. We report one-step intervention and counterfactual MSE on held-out trajectories, structural Hamming distance (SHD), and for the agent tasks the safety-violation rate (fraction of all episodes in which the executed action was unsafe), goal rate, appropriate deferral (fraction of no-safe episodes in which the agent deferred), and unnecessary deferral (fraction of episodes with a safe action in which it deferred). On ER-10, main models use 5 seeds and ablations 3 to 5 seeds; on the other graphs we use 3 seeds, and the baseline and original-model numbers there are the first-version runs (same code and data). All values are mean ± sample standard deviation over seeds.

	Chain	Fork	Collider	ER-10	ER-20
Intervention MSE					
Small Transformer	0.046 ± 0.002	0.033 ± 0.002	0.031 ± 0.000	0.037 ± 0.002	0.071 ± 0.002
Vanilla VAE	0.031 ± 0.000	0.030 ± 0.001	0.025 ± 0.000	0.028 ± 0.000	0.048 ± 0.001
Dreamer-style	0.032 ± 0.001	0.029 ± 0.001	0.025 ± 0.000	0.028 ± 0.000	0.048 ± 0.000
AC-LSCM (original)	0.101 ± 0.004	0.057 ± 0.001	0.046 ± 0.002	0.058 ± 0.004	0.221 ± 0.009
AC-LSCM (simplified)	$\mathbf{0.029 \pm 0.000}$	$\mathbf{0.028 \pm 0.001}$	$\mathbf{0.025 \pm 0.001}$	$\mathbf{0.027 \pm 0.000}$	$\mathbf{0.046 \pm 0.001}$
Counterfactual MSE					
Small Transformer	0.049 ± 0.002	0.034 ± 0.002	0.032 ± 0.001	0.039 ± 0.001	0.077 ± 0.002
Vanilla VAE	0.032 ± 0.000	0.030 ± 0.001	0.025 ± 0.000	0.029 ± 0.001	0.050 ± 0.000
Dreamer-style	0.034 ± 0.000	0.029 ± 0.001	0.024 ± 0.001	0.029 ± 0.001	0.051 ± 0.001
AC-LSCM (original)	0.064 ± 0.001	0.177 ± 0.037	0.052 ± 0.006	0.127 ± 0.040	0.130 ± 0.015
AC-LSCM (simplified)	$\mathbf{0.021 \pm 0.000}$	$\mathbf{0.019 \pm 0.000}$	$\mathbf{0.019 \pm 0.001}$	$\mathbf{0.020 \pm 0.000}$	$\mathbf{0.036 \pm 0.001}$
Structural Hamming distance (true edges in parentheses)					
AC-LSCM (original)	9.0 ± 0.0 (9)	9.0 ± 0.0 (9)	9.0 ± 0.0 (9)	8.0 ± 0.0 (8)	56.0 ± 0.0 (56)
AC-LSCM (simplified)	0.0 ± 0.0 (9)	2.0 ± 2.6 (9)	7.0 ± 1.0 (9)	1.0 ± 0.0 (8)	43.7 ± 0.6 (56)

Table 2: One-step prediction error on held-out data (lower is better; best in bold) and structure recovery. The original AC-LSCM learned an empty graph on every graph type and seed: its SHD equals the number of true edges with zero variance. The simplified AC-LSCM has the lowest intervention and counterfactual error on all five graphs. ER-10 numbers are from this version's runs; the baseline and original-model numbers on the other graphs are from the first version.

9 Results

9.1 Diagnosis: the original model learned no graph

Table 2 shows that the original AC-LSCM's SHD equals the number of true edges on every graph, with zero variance across seeds; in every run the thresholded adjacency is empty. The first version read SHD = 8 on ER-10 as "the learned graph is precise"; it is the SHD of the empty graph. Three causes combine. The adjacency logits start at 0, i.e. exactly at the evaluation threshold; the ℓ_1 and NOTEARS penalties (on a graph where acyclicity is automatic) push every logit down; and because training uses a soft relaxed mask, the transition networks can partially compensate for a shrinking edge, so its loss gradient never wins against the penalty. With an empty graph each factor's next value is predicted from the action alone, ignoring the current state. This explains several first-version observations: intervention MSE twice that of the baselines; the "DAG constraint converged exactly to zero"; ablating the graph loss (SHD 68.7, i.e. edges switched on) halving the error; and the do-operator ablation changing almost nothing, since with no parents the do-operator has nothing to sever. It also explains the counterfactual "fingerprint" reported there: when the transition ignores the state, the abducted residual $\hat{\epsilon}_t$ carries the state-dependent part of $\mathbf{z}_{t+1}$, so abduction partially repairs the factual prediction.

The simplified model removes all three causes. It has the lowest intervention and counterfactual error on all five graphs, with the clearest margin on counterfactuals (for example 0.020 versus 0.029 on ER-10). Structure recovery is exact on the chain, nearly exact on ER-10 (one missing edge on every seed) and the fork, and poor on the collider and ER-20, where many weak edges point into the same node; all errors are missing edges, and no run learned a spurious edge.

9.2 The direct-effect task does not test consequence modeling

Table 3 re-runs the first version's agent task. Under its original protocol we reproduce the headline: the Transformer violates safety in 15.6% of episodes and the original AC-LSCM in under 1% when its predictions are decoded. Two observations change what this means. First, the MLP-based

	Original, latent predicates		Original, decoded		In-range, decoded	
Model	Violations	Approp. defer	Violations	Approp. defer	Violations	Approp. defer
Small Transformer	–	–	0.156 ± 0.052	0.220 ± 0.259	0.020 ± 0.012	0.900 ± 0.061
Vanilla VAE	0.242 ± 0.083	0.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000
Dreamer-style	0.504 ± 0.417	0.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000
AC-LSCM (original)	0.072 ± 0.161	0.820 ± 0.402	0.008 ± 0.018	0.960 ± 0.089	0.000 ± 0.000	1.000 ± 0.000
AC-LSCM (simplified)	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000

Table 3: Direct-effect task (ER-10, 5 seeds). "Original, latent" is the protocol of the first version, where predicates read the predicted latent; the Transformer has no latent and is evaluated on its predicted observation. With decoded predictions every model except the Transformer solves the task; with in-range action values the Transformer's violations also drop to 2%.

Model	Violations ↓	Goal rate ↑	Approp. defer ↑	Unneeded defer ↓
Small Transformer	0.102 ± 0.013	0.684 ± 0.018	0.656 ± 0.048	0.004 ± 0.003
Vanilla VAE	0.073 ± 0.016	0.709 ± 0.012	0.752 ± 0.059	0.005 ± 0.002
Dreamer-style	0.118 ± 0.025	0.717 ± 0.007	0.597 ± 0.083	0.006 ± 0.004
AC-LSCM (original)	0.321 ± 0.018	0.659 ± 0.025	0.000 ± 0.000	0.000 ± 0.000
AC-LSCM (simplified)	0.084 ± 0.042	0.705 ± 0.004	0.685 ± 0.163	0.015 ± 0.004
AC-LSCM (simplified, $\lambda_1 = 0$)	0.047 ± 0.003	0.714 ± 0.004	0.849 ± 0.015	0.004 ± 0.004
AC-LSCM (simplified, $\lambda_1 = 0$, no $\mathcal{L}_{\text{cf}}$)	0.069 ± 0.009	0.727 ± 0.003	0.747 ± 0.034	0.003 ± 0.002
AC-LSCM (simplified, true graph)	0.048 ± 0.002	0.717 ± 0.003	0.827 ± 0.000	0.006 ± 0.007
Oracle: true SCM, noise-free	0.047	0.683	0.853	0.027

Table 4: Consequence task (ER-10, 300 episodes, horizon 3; goal-rate ceiling 0.75; 25% of episodes have no safe action). 5 seeds for the main models, 3 for the $\lambda_1 = 0$ and true-graph variants. The oracle plans with the noise-free true SCM from the noisy observation.

baselines (Vanilla VAE, Dreamer-style) also have zero violations once their predictions are decoded. Under the first version's latent-space protocol they fail, because their latents are not aligned with the observation coordinates on which the thresholds are defined; the "passive policy" seeds of the original AC-LSCM in the first version have the same cause and disappear in decoded space (goal rate 0.80 on every seed). Second, when the unsafe values are moved inside the training range, the Transformer's violation rate falls from 15.6% to 2.0%. Its failure on the original task is a failure to extrapolate action values beyond the training range, not a failure to represent interventions. The direct-effect task is solved by any model that learns $f_i(\varnothing, v) \approx v$ and extrapolates monotonically, including the original AC-LSCM, which sees no state at all. It cannot support the first version's $36\times$ claim, which we withdraw.

9.3 Consequence task

Table 4 reports the consequence task. Three results stand out.

The original AC-LSCM is a graph-blind planner. Its violation rate of 32.1% matches the 31.7% of the reference planner that knows only each action's direct effect, and it never defers. This is the expected behaviour of a model with an empty graph, and it is roughly three times worse than every baseline.

With a correct graph, AC-LSCM plans at the level of the oracle. With the true graph fixed, or with no sparsity penalty at all ($\lambda_1 = 0$, so all 90 possible edges are kept and each f_i learns which inputs to ignore), the simplified model violates safety in 4.8% and 4.7% of episodes, matching the noise-free oracle (4.7%) and about a third below the best baseline (Vanilla VAE, 7.3%). It does so without giving up goal rate (0.71 to 0.72 against a ceiling of 0.75) and with few unnecessary

AC-LSCM variant	Seeds	Int MSE	CF MSE	Edges	SHD	Violations	Goal rate
Original	5	0.058 ± 0.004	0.127 ± 0.040	0.0 ± 0.0	8.0 ± 0.0	0.321 ± 0.018	0.659 ± 0.025
Original, no do-operator	3	0.060 ± 0.004	0.141 ± 0.034	0.0 ± 0.0	8.0 ± 0.0	0.319 ± 0.016	0.663 ± 0.012
Simplified	5	0.027 ± 0.000	0.020 ± 0.000	7.0 ± 0.0	1.0 ± 0.0	0.084 ± 0.042	0.705 ± 0.004
Simplified, no do-operator	3	0.027 ± 0.001	0.021 ± 0.001	7.0 ± 0.0	1.0 ± 0.0	0.091 ± 0.048	0.706 ± 0.002
Simplified, no $\mathcal{L}_{\text{cf}}$	3	0.030 ± 0.002	0.028 ± 0.001	4.3 ± 1.2	3.7 ± 1.2	0.280 ± 0.029	0.666 ± 0.030
Simplified, soft mask	4	0.030 ± 0.001	0.020 ± 0.000	5.0 ± 0.0	3.0 ± 0.0	0.246 ± 0.007	0.709 ± 0.006
Simplified, $\lambda_1 = 0$	3	0.027 ± 0.000	0.019 ± 0.001	90.0 ± 0.0	82.0 ± 0.0	0.047 ± 0.003	0.714 ± 0.004
Simplified, $\lambda_1 = 0$, no $\mathcal{L}_{\text{cf}}$	5	0.027 ± 0.000	0.021 ± 0.001	90.0 ± 0.0	82.0 ± 0.0	0.069 ± 0.009	0.727 ± 0.003
Simplified, true graph	3	0.027 ± 0.000	0.019 ± 0.001	8.0 ± 0.0	0.0 ± 0.0	0.048 ± 0.002	0.717 ± 0.003

Table 5: Ablations on ER-10 (8 true edges). The last two columns are the consequence task. "Soft mask" is the simplified model before change (v), i.e. with a relaxed rather than straight-through adjacency sample.

deferrals.

Part of that advantage comes from privileged supervision. The counterfactual loss of Equation (7) regresses predicted latents onto ground-truth counterfactual *latents*, which tells the model the true state coordinates; the baselines receive no such signal. Without it, the dense model reaches $6.9\% \pm 0.9\%$ violations: comparable to the VAE ($7.3\% \pm 1.6\%$), better than the Transformer (10.2%) and Dreamer-style (11.8%), and with the highest goal rate of all models (0.727). The fair architectural comparison therefore shows a modest advantage at best on this task, while the counterfactual-error advantage of Table 2 persists without the privileged loss (0.021 against 0.029 for the best baseline; Table 5).

The simplified model with its default sparsity ($\lambda_1 = 0.01$) sits in between, at $8.4\% \pm 4.2\%$. Every one of its five seeds recovers seven of the eight true edges and misses the same one, $z^0 \to z^9$: one of the two weakest edges in the graph (weight 0.65), and one that feeds the safety factor. A structure learner that errs towards missing weak edges produces a forward model that is confidently wrong about exactly the indirect effects the safety check needs.

9.4 Ablations

Table 5 isolates the components.

The do-operator has no measurable effect. Removing it changes neither prediction error nor consequence-task safety, in the original model (where there are no parents to sever) or in the simplified one (8.4% versus 9.1% violations, well within seed variation). This retracts the first version's claim that the do-operator "carries the result". It is consistent with Section 3: when the action targets a known factor and is given to that factor's mechanism as an input, the network can learn to ignore the parents from data, and explicit severing adds nothing measurable in this setting.

The mask relaxation decides what structure is learned. With the soft relaxed mask, the simplified model misses three edges ($2 \to 7$, $0 \to 9$, $5 \to 9$) and violates safety in 24.6% of episodes; with the straight-through mask it misses one and reaches 8.4%. Prediction error barely moves (0.030 versus 0.027), so held-out MSE is a weak guide to the quality of the learned structure for planning.

Sparsity trades structure recovery against safety. Removing the sparsity penalty gives a fully dense graph (SHD 82) with the same prediction error and the best safety. On this task an interpretable sparse graph is not needed for planning, and pruning is harmful when it removes weak true edges.

Counterfactual supervision matters, and it is privileged. Without $\mathcal{L}_{\text{cf}}$ the sparse model loses more edges (SHD 3.7) and its violation rate rises to 28.0%; the dense model without it reaches 6.9%. In both cases the counterfactual loss is helping by anchoring the latent coordinates, which a real agent would not get for free.

10 Related Work

Latent world models and planning. Ha and Schmidhuber [6] established the template of learning a compressed latent dynamics model and acting through it. PlaNet [7] planned directly in a learned latent space, the Dreamer line [8] scaled latent imagination to diverse domains, and MuZero [18] showed that a learned model is sufficient for strong planning. These models treat the action as an input to a monolithic transition. AC-LSCM instead factorizes the transition per latent factor over a learned sparse graph and treats actions as interventions that replace individual mechanisms.

Causal structure in dynamics models. Causal Dynamics Learning [19] learns a sparse causal graph between state variables and actions in order to obtain task-independent state abstractions, and Ke et al. [11] systematically evaluate causal-discovery-based world models in model-based reinforcement learning. The simplified AC-LSCM is close in spirit to these works: a sparse, lagged graph over state factors learned jointly with per-factor mechanisms.

Causal representation learning. Schölkopf et al. [17] survey the program of learning representations aligned with causal mechanisms. CITRIS [13] proves identifiability of causal factors from temporal sequences with known intervention targets, which is the regime of our experiments; mechanism sparsity [12] and weak supervision from paired interventions [3] provide related guarantees. We do not contribute identifiability results; our encoder is a plain MLP and our latents are anchored by the known action targets.

Continuous DAG learning. NOTEARS [20] reformulated DAG learning as continuous optimization with a smooth acyclicity function, and DAGMA [2] improved its numerical behaviour with a log-determinant characterization. Our negative result is specific to how these penalties were combined with a relaxed adjacency and a sparsity term in a lagged model: there, the acyclicity term is unnecessary and, with the original weights, the combined penalty removed every edge.

Safe action selection. Shielding [1] blocks actions whose consequences would violate a formal safety specification, using a model of the environment. Algorithm 1 is a learned, approximate analogue: the shield is the safety predicate evaluated on the forward model's simulated trajectory, with deferral when no action passes.

World models and risk for LLM agents. RAP [9] uses an LLM as its own world model for planning, and WebDreamer [5] uses an LLM to simulate the outcome of candidate web actions before executing them. ToolEmu [16] uses an LM-emulated sandbox to identify risky agent actions. These systems obtain the forward model from the language model itself; our question is what structure a forward model needs in order to get the consequences of interventions right, studied in a setting where ground truth is available. CLadder [10] and CounterBench [4] document weaknesses of current LLMs on formal causal queries and motivate the problem, but we have not evaluated on them.

11 Limitations and Open Questions

Synthetic, low-dimensional setting. All results are on synthetic SCMs with $K \leq 20$ scalar factors, observations that are a noisy copy of the latent state, single-factor actions with known targets, and small MLPs. The ground-truth mechanisms are additive-noise and lagged, which is

exactly the model class of the simplified AC-LSCM. Whether the gains survive richer observations, unknown intervention targets, instantaneous effects, or misspecified mechanisms is untested.

No language-domain evaluation. We have not connected AC-LSCM to a language model or evaluated it on text-based agent tasks or causal-reasoning benchmarks. The paper's motivation concerns LLM agents; its evidence concerns forward models in synthetic environments. Bridging that gap is the most important open step.

Counterfactual supervision. The counterfactual loss requires ground-truth counterfactual latents, which are available only in simulation and give AC-LSCM information about the true state coordinates that the baselines do not receive. Without it, AC-LSCM's consequence-task safety is on par with the best baseline (Sections 9.3 and 9.4).

Task design. The consequence task uses one hand-chosen safety factor and one goal factor on one graph, a fixed horizon of three steps, and candidate sets constructed so that a fixed fraction of episodes has no safe action. Candidates whose realized outcome falls within a margin of a threshold are excluded so that labels are decided by the action rather than by noise. Other choices would give different absolute numbers.

Evaluation protocol. Early stopping uses a validation split taken from the training trajectory. The simplified model's settings came from the first-version ablations, one short pilot run, and one further change (the straight-through mask) chosen on prediction error after its first full runs; no setting was tuned on the agent tasks, and the baselines keep the hyperparameters of the first version. The agent-task configurations themselves (thresholds, horizon, margin, candidate counts) were fixed once using only the oracle and two model-free reference predictors.

12 Conclusion

Agents that act on real systems should predict what their actions will cause before acting. We studied a structured forward model for this purpose and, re-examining an earlier version of this work, found that its original specification learned no causal structure and that its headline agent result measured extrapolation of action values rather than modeling of consequences. With those problems fixed, the evidence is narrower and, we believe, more useful. A factorized, intervention-aware forward model gives the best counterfactual predictions in our synthetic setting. In planning, what matters is whether the model captures the indirect paths that lead to unsafe states: a model that keeps all true edges matches an oracle, a model that prunes one weak edge into the safety factor loses much of that advantage, and the explicit do-operator contributes nothing measurable. Without ground-truth counterfactual supervision, the architecture is on par with a well-tuned unstructured latent model. For safety-critical use this argues for conservative (denser) structure and for evaluation protocols that test downstream consequences, in range of the training data, with every model judged in the same space. Connecting such forward models to language-model agents remains future work.

Code and Reproducibility

Code, configurations, per-seed result files, and the scripts that produce every table are available at `https://github.com/mplusm/AC-LSCM-Outcome-Aware-Research-Paper`. The first-version

experiments ran on a single NVIDIA Tesla T4 in fp32 (under 50 GPU-hours). The experiments added in this version ran on a 4-core CPU. All runs use deterministic seeding.

Acknowledgments

The author thanks the open research community working on causal representation learning, world models, and agent safety, whose published work and benchmarks made this study possible.

References

[1] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu. Safe reinforcement learning via shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. arXiv:1708.08611.

[2] K. Bello, B. Aragam, and P. Ravikumar. DAGMA: Learning DAGs via M-matrices and a log-determinant acyclicity characterization. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[3] J. Brehmer, P. de Haan, P. Lippe, and T. Cohen. Weakly supervised causal representation learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022. arXiv:2203.16437.

[4] Y. Chen, V. K. Singh, J. Ma, and R. Tang. CounterBench: Evaluating and improving counterfactual reasoning in large language models. arXiv:2502.11008, 2025.

[5] Y. Gu, K. Zhang, Y. Ning, B. Zheng, B. Gou, T. Xue, C. Chang, S. Srivastava, Y. Xie, P. Qi, H. Sun, and Y. Su. Is your LLM secretly a world model of the internet? Model-based planning for web agents. arXiv:2411.06559, 2024.

[6] D. Ha and J. Schmidhuber. World models. arXiv:1803.10122, 2018.

[7] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2019. arXiv:1811.04551.

[8] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. arXiv:2301.04104, 2023.

[9] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu. Reasoning with language model is planning with world model. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.14992.

[10] Z. Jin, Y. Chen, F. Leeb, L. Gresele, O. Kamal, Z. Lyu, K. Blin, F. Gonzalez Adauto, M. Kleiman-Weiner, M. Sachan, and B. Schölkopf. CLadder: Assessing causal reasoning in language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[11] N. R. Ke, A. Didolkar, S. Mittal, A. Goyal, G. Lajoie, S. Bauer, D. Rezende, Y. Bengio, M. Mozer, and C. Pal. Systematic evaluation of causal discovery in visual model based reinforcement learning. arXiv:2107.00848, 2021.

[12] S. Lachapelle, P. Rodríguez López, Y. Sharma, K. Everett, R. Le Priol, A. Lacoste, and S. Lacoste-Julien. Disentanglement via mechanism sparsity regularization: A new principle for nonlinear ICA. In *Conference on Causal Learning and Reasoning*, 2022. arXiv:2107.10098.

[13] P. Lippe, S. Magliacane, S. Löwe, Y. M. Asano, T. Cohen, and E. Gavves. CITRIS: Causal identifiability from temporal intervened sequences. In *International Conference on Machine Learning*, 2022. arXiv:2202.03169.

[14] M. Madapathi. Outcome-aware agents: From token prediction to action consequence modeling (first version). Zenodo, 2026. `https://doi.org/10.5281/zenodo.20379456`.

[15] J. Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

[16] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024. arXiv:2309.15817.

[17] B. Schölkopf, F. Locatello, S. Bauer, N. R. Ke, N. Kalchbrenner, A. Goyal, and Y. Bengio. Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634, 2021.

[18] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[19] Z. Wang, X. Xiao, Z. Xu, Y. Zhu, and P. Stone. Causal dynamics learning for task-independent state abstraction. In *International Conference on Machine Learning*, 2022. arXiv:2206.13452.

[20] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing. DAGs with NO TEARS: Continuous optimization for structure learning. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

A Experimental Details

Data. Each environment is an SCM over K scalar factors with lagged additive-noise mechanisms $z^i_{t+1} = \tanh\left(\sum_j W_{ij} z^j_t\right) + \epsilon^i_t$, $\epsilon^i_t \sim \mathcal{N}(0, 0.1^2)$, with edge weights drawn uniformly from $\pm[0.5, 2]$. Observations are $\mathbf{x}_t = \mathbf{z}_t + \mathcal{N}(0, 0.1^2)$. Graphs: chain, fork, and collider with $K = 10$, and Erdős–Rényi graphs with edge probability 0.3 for $K = 10$ (8 edges) and $K = 20$ (56 edges). Each step applies, with probability 0.5, one intervention $\mathrm{do}(z^i = v)$ with i uniform and $v \sim U(-1.5, 1.5)$. Each training set is one trajectory of $N = 10{,}000$ steps; each test set is a separate trajectory of 500 steps. Counterfactual targets reuse the same noise with a different random single-factor action.

Models. All latent models share an encoder and decoder (two-layer MLPs, hidden size 128) with a K-dimensional latent. AC-LSCM uses one transition MLP per factor (input $K+1$, hidden size 32) and a Gumbel-sigmoid adjacency with temperature 0.5. Vanilla VAE uses a single MLP transition (hidden 128) on the concatenation of latent and action; Dreamer-style uses a GRU cell with hidden size K. The Transformer embeds each of the $2K$ scalars of $(\mathbf{x}_t, a_t)$ as a token ($d = 128$, 2 pre-norm layers, 4 heads, feed-forward 512), mean-pools, and predicts $\mathbf{x}_{t+1}$ with an MLP head. Missing action components are encoded as 0 for all baselines.

Optimization. Adam with learning rate 10^{-3}, batch size 64, gradient clipping at 1.0, up to 200 epochs with early stopping (patience 20) on the validation loss. Loss weights $\beta = (1, 1, 0.5, 1)$; the graph-loss weight is ramped linearly from 0 over the first 50 epochs. Original AC-LSCM: $\lambda_1 = 0.1$, $\lambda_2 = 1$ (NOTEARS), adjacency logits initialized at 0, hinge margin $\gamma = 1$. Simplified AC-LSCM: $\lambda_1 = 0.01$, $\lambda_2 = 0$, logits initialized at $+2$, no hinge, exact abduction, straight-through hard Gumbel-sigmoid adjacency samples.

Compute. The experiments of this version (61 training runs) ran on a shared 4-core CPU without a GPU; a run of AC-LSCM on ER-10 takes roughly 15 to 45 minutes there, and a run of the Transformer roughly 90 minutes.

Metrics. Intervention MSE is the observation-space error of the one-step prediction under the logged action on the test trajectory. Counterfactual MSE is the error of the counterfactual prediction against the ground-truth counterfactual observation. Structural Hamming distance (SHD) counts mismatched entries between the thresholded learned adjacency and the true one. All tables report the mean and sample standard deviation over seeds.

B Relation to the First Version

The first version [14] reported, on the direct-effect task with predicates evaluated in latent space, a safety-violation rate of 0.005 ± 0.019 for AC-LSCM over 13 seeds (8 trained on N=10,000 samples and 5 on N=2,000, pooled) against 0.180 ± 0.022 for the Transformer over 3 seeds, and attributed the gain to the do-operator and abduction. This version re-runs that task with the same code and finds: (i) the original model's graph is empty in every run, so its predictions do not depend on the state; (ii) the four "failed" seeds of the first version (three "passive" and one "mis-ranked") are an artifact of thresholding latents that are not aligned with observation coordinates and disappear when predictions are decoded; (iii) the Transformer's violations come from extrapolating action values outside the training range; and (iv) the do-operator ablation has no effect. The ablation conclusions of the first version that the hinge term and the NOTEARS term should be removed are confirmed; the claims about the do-operator, about precise structure recovery (SHD 8), and about the 36× safety improvement are withdrawn. The first-version per-seed results remain in the repository under `results/`; this version's are under `results_v2/`.